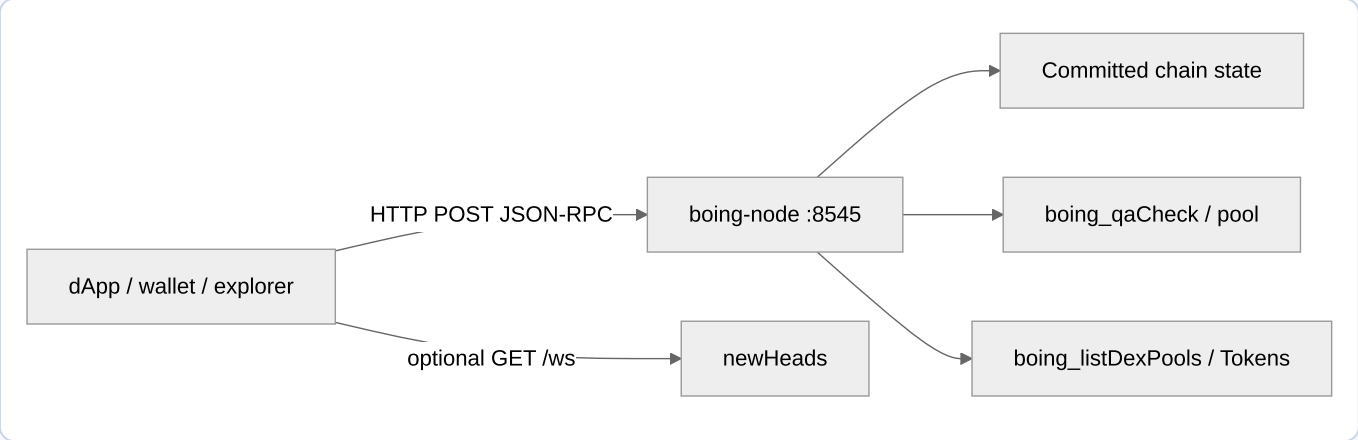
Boing Network — JSON-RPC API Specification

> 👋 **Everyday users:** you do not call this API directly. The wallet and explorer do it for you.
> 🛠️ **Developers:** this is the contract. Prefer `boing-sdk createClient(url)` over hand-rolled `fetch`. Probe with `boing_getRpcMethodCatalog` / `probeBoingRpcCapabilities`.
> 🚦 **Operators:** CORS, batch limits, `/live` `/ready`, and `BOING_*` env for `boing_getNetworkInfo.end_user`.

Version: 0.1
Transport: HTTP POST
Encoding: JSON-RPC 2.0
References: RUNBOOK.md · BOING-RPC-ERROR-CODES-FOR-DAPPS.md



Public testnet: `https://testnet-rpc.boing.network/`. Local default: `http://127.0.0.1:8545/`.

If `boing_chainHeight` works but newer methods return `-32601`, the process is an **older binary** or a gateway allowlist — upgrading the `boing-node` binary on your laptop does **not** change the public explorer. See THREE-CODEBASE-ALIGNMENT.md §2.1.

Overview

Boing nodes expose a JSON-RPC HTTP interface for submitting transactions, querying chain state, and simulation. Rate limiting applies per `RateLimitConfig` (see SECURITY-STANDARDS.md). When enabled, **one HTTP `POST /` consumes one token** even if the body is a **JSON-RPC batch** (array of requests). Responses use HTTP **429** with header `Retry-After: 1` (second) and a JSON-RPC error body (`-32016`) so clients can backoff without parsing the body. The same JSON body may include a sibling `boing_http` object (`code` , `message` , `request_id`) for log correlation.

Diagnosing `-32601` (method not found)

If `boing_chainHeight` works but newer methods (e.g. `boing_getSyncState` , `boing_getNetworkInfo` , `boing_getLogs` , `boing_getTransactionReceipt`) return `-32601` **Method not found**, the HTTP endpoint is **not** running the same RPC surface as **current** `boing-node` in this repository: the process is usually an **older binary** or traffic is going through a **gateway that only whitelists some method names**. Rebuild and restart the node from this repo (`cargo build -p boing-node --release` , see RUNBOOK.md), or point the client at the node's JSON-RPC port with no filtering.

Use `boing-sdk` `probeBoingRpcCapabilities` / `npm run probe-rpc` (from `boing-sdk` after `npm run build` , or from the repo root) — the printed JSON includes `clientVersion` / `supportedMethodCount` when `boing_clientVersion` / `boing_rpcSupportedMethods` exist; the `diagnosis` field explains gaps when `explainBoingRpcProbeGaps` applies. For a single pass that also checks `GET /openapi.json` , `/.well-known/boing-rpc` , and `/live.json` , run `npm run boing:smoke` from the repo root (after `npm run build` in `boing-sdk`) or call `doctorBoingRpcEnvironment` in code.

Base URL

```
http://<host>:<rpc_port>/
```

Default RPC port: `8545` .

The same HTTP listener serves `GET /live` , `GET /ready` , `GET /ws` , `GET /openapi.json` , `GET /.well-known/boing-rpc` , and `POST /` (JSON-RPC). `GET /` (root) returns **405 Method Not Allowed** with `Allow: GET, HEAD, POST, OPTIONS` and a short plain-text hint so accidental browser or `curl` visits fail clearly. `HEAD /` returns the same **405** and `Allow` with an empty body. `OPTIONS /` advertises the same allowed methods. The in-tree handler returns **204** with no body; the **CORS** middleware may respond to browser preflights with **200** before the route runs—either way, integrators can rely on `Allow` for discovery.

JSON probes: `GET /live.json` (or `GET /live` with `Accept: application/json`) returns `{ "ok": true }` . `GET /ready.json` (or `Accept: application/json` on `/ready`) returns `{ "ready": true }` , or `503` with JSON `{ "ready": false, "reason": "peers_below_min", "peers", "min_peers" }` when

`BOING_RPC_READY_MIN_PEERS` is set and peers are below the threshold (`Retry-After: 5` on **503**). `HEAD /live` and `HEAD /ready` return status only.

HTTP OpenAPI: `GET /openapi.json` returns the same OpenAPI 3.1 object as `boing_getRpcOpenApi`. `GET /.well-known/boing-rpc` returns a small JSON document with `schema_version` and path hints (no JSON-RPC round-trip required).

Request correlation: Clients may send `X-Request-Id` on any request; the node echoes it on the response. If omitted, the node assigns a UUID. Browsers reading this header from cross-origin responses need CORS `Access-Control-Expose-Headers` (the node exposes `x-request-id`, `retry-after`, and `x-boing-rpc-version`).

Response headers: Successful JSON-RPC responses include `X-Boing-RPC-version` (e.g. `boing-node/0.1.0`) so proxies and clients can detect binary skew without an extra `boing_clientVersion` call. The same header is set on `/live` and `/ready`.

Request body limit: `POST /` JSON bodies default to **8 MiB** max (large signed-transaction hex). Set `BOING_RPC_MAX_BODY_MB` (integer ≥ **1**) to override. Oversized bodies receive HTTP **413** with `Content-Type: application/json` and body `{ "code": "payload_too_large", "message", "request_id" }` when `x-request-id` is available on the response.

JSON-RPC batch: The body may be a **JSON array** of request objects. The HTTP response is a **JSON array** of response objects in the same order. Requests **without an** `id` are treated as **notifications** and produce **no entry** in the batch response. If **every** entry in the batch is a notification, the server returns **HTTP 204** with **no body** (per JSON-RPC 2.0). A **single** notification (non-batch) also returns **204**. An empty batch array `[]` returns `200` with body `[]`. Max batch length defaults to **32** and is capped at **256**; set `BOING_RPC_MAX_BATCH` to override (invalid values fall back to the default).

Browser CORS

`boing-node` sends **CORS** headers for a built-in allowlist (Boing web apps and common local dev ports). To allow **additional** browser origins without recompiling, set environment variable `BOING_RPC_CORS_EXTRA_ORIGINS` to a comma-separated list (e.g. `http://localhost:9999,https://preview.example.com`). Invalid entries are skipped with a warning in node logs.

Chain tip, committed height, and finality

Boing uses a **HotStuff-style BFT** consensus layer (`boing-consensus`): a block is appended to this node's `ChainState` only after it is **committed** through that path. There is **no separate JSON-RPC "unsafe head"** ahead of commit in the current node implementation.

Term	Meaning in this codebase
`boing_chainHeight` / `head_height` in `boing_getSyncState`	Height of the latest **committed** block stored on this node (the chain tip).
Finalized (protocol sense)	Under standard BFT assumptions (honest quorum, etc.), a committed block is not reverted by consensus.
`finalized_height` in `boing_getSyncState`	Today **equal to** `head_height`, because the node only exposes committed blocks. Reserved so a future release can report lag (e.g. optimistic execution vs commit) without breaking clients.
Syncing / lagging peers	A node that is still catching up may report a **lower** height than the rest of the network until it imports commits. Clients should not treat “height stopped increasing” as finality across the whole network without comparing multiple sources.

Wallets and observers that need a single number for “how deep is my tx” can use `boing_chainHeight` or `boing_getSyncState`; until multiple heights diverge in a future version, they are equivalent for “committed tip.”

dApp network discovery

`boing_getNetworkInfo` (params `[]`) returns a single JSON object for wallets and dApps: committed tip fields (same as `boing_getSyncState`), target block time, `client_version`, optional `chain_id` / `chain_name` from environment (see below), consensus summary (`validator_count`, `model`), `native_currency`, `chain_native` (chain-wide sums over the committed account table), `developer` (doc URLs, npm package id, WebSocket `/ws` handshake, and API discovery method names), and an explicit `rpc.not_available` list.

`chain_native`: `account_count`, `total_balance`, `total_stake`, and `total_native_held` are **u128 decimal strings** (same style as `boing_getAccount`). They are the **sum** of per-account `balance` / `stake` in this node’s committed state — **not** “circulating supply” or “total minted”. `as_of_height` is the committed tip height those sums match.

`treasury`: Canonical protocol treasury account `0x5452454153555259000000000000000000000000000000000000000000000001` (`PROTOCOL_TREASURY`), current `balance`, and the effective `fee_policy` (same object as `boing_getNetworkFeePolicy`). Native tx fees default to **100% treasury**; mining (block emission) still credits the round proposer.

Important: Public JSON-RPC still does **not** expose **staking APY** or a protocol-defined **circulating supply**. The node lists remaining gaps under `rpc.not_available` (today `staking_apy`) and `rpc.not_available_note`. **Per-account** balance and stake remain available via `boing_getAccount`.

Operator environment (optional):

Variable	Effect on `boing_getNetworkInfo`
`BOING_CHAIN_ID`	If set to a decimal integer (e.g. `6913` for public testnet), included as JSON number `chain_id`. If unset or invalid, `chain_id` is `null` — clients must use their configured chain id (e.g. from app env).
`BOING_CHAIN_NAME`	Optional human-readable `chain_name` (e.g. `Boing Testnet`). Omitted when unset.
`BOING_CHAIN_DISPLAY_NAME`	Optional wallet-facing display string in `end_user.chain_display_name` (does not replace `chain_name`).
`BOING_EXPLORER_URL`	Optional block explorer base URL in `end_user.explorer_url`.
`BOING_FAUCET_URL`	Optional faucet URL in `end_user.faucet_url`.
`BOING_CANONICAL_NATIVE_CP_POOL`	Optional 32-byte `AccountId` hex (with or without `0x`) in `end_user.canonical_native_cp_pool` — public RPC operators publish the canonical native constant-product pool so dApps need not hardcode it (OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md). Invalid values are ignored with a warning in node logs.
`BOING_CANONICAL_NATIVE_DEX_FACTORY`	Optional pair-directory contract `AccountId` in `end_user.canonical_native_dex_factory` for default `boing_getContractStorage` / `boing_getLogs` wiring.
`BOING_CANONICAL_NATIVE_DEX_MULTIHOP_SWAP_ROUTER`	Optional multihop (2–6 pool) swap router `AccountId` in `end_user.canonical_native_dex_multihop_swap_router`.
`BOING_CANONICAL_NATIVE_DEX_LEDGER_ROUTER_V2`	Optional ledger forwarder `AccountId` (**160**-byte inner calldata) in `end_user.canonical_native_dex_ledger_router_v2`.
`BOING_CANONICAL_NATIVE_DEX_LEDGER_ROUTER_V3`	Optional ledger forwarder `AccountId` (**192**-byte inner calldata) in `end_user.canonical_native_dex_ledger_router_v3`.
`BOING_CANONICAL_NATIVE_AMM_LP_VAULT`	Optional LP vault `AccountId` in `end_user.canonical_native_amm_lp_vault`.
`BOING_CANONICAL_NATIVE_LP_SHARE_TOKEN`	Optional LP share token `AccountId` in `end_user.canonical_native_lp_share_token`.

`rpc_surface` on `boing_getNetworkInfo`: mirrors `boing_health.rpc_surface` (batch cap, body limit, `boing_getLogs` bounds, WS cap, rate limit, ready peer gate).

Request Format

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "boing_chainHeight",
  "params": []
}
```

Response Format

Success:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": 42
}
```

Error:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid params: expected [hex_signed_tx]"
  }
}
```

Methods

Method index (`boing_rpcSupportedMethods`)

Current `boing-node` implements these 66 JSON-RPC methods (same set returned by `boing_rpcSupportedMethods`, sorted alphabetically):

`boing_chainHeight`, `boing_checkNames`, `boing_clientVersion`, `boing_faucetRequest`, `boing_getAccount`, `boing_getAccountProof`, `boing_getAccountTransactions`, `boing_getBalance`, `boing_getBlob`, `boing_getBlobAccess`, `boing_getBlobDasLightClientBatch`, `boing_getBlobDasSample`, `boing_getBlobDasSuccinctProofPlan`, `boing_getBlobMeta`, `boing_getBlobSampleServeChallenge`, `boing_getBlockByHash`, `boing_getBlockByHeight`, `boing_getContractCode`, `boing_getContractStorage`, `boing_getDexToken`, `boing_getLeaderVrfOutput`, `boing_getLogs`, `boing_getMediaBlobParams`, `boing_getNameInfo`, `boing_getNetworkFeePolicy`, `boing_getNetworkInfo`, `boing_getQaRegistry`, `boing_getRpcMethodCatalog`, `boing_getRpcOpenApi`, `boing_getSyncState`, `boing_getTokenMetadata`, `boing_getTransactionReceipt`, `boing_getValidators`, `boing_health`, `boing_listBlobSampleOffers`, `boing_listBlobSampleServeChallenges`, `boing_listBlsDualKeys`, `boing_listDexPools`, `boing_listDexTokens`, `boing_listSlashRecords`, `boing_listSmtBlobSampleOffers`, `boing_listTokens`, `boing_matchBlobSampleOffer`, `boing_namesOwnedBy`, `boing_operatorApplyFeePolicy`, `boing_operatorApplyQaPolicy`, `boing_putBlobSampleOffer`, `boing_qaCheck`, `boing_qaPoolConfig`, `boing_qaPoolList`, `boing_qaPoolVote`, `boing_quoteBlobSampleSettle`, `boing_registerBlsDualKey`, `boing_registerDappMetrics`, `boing_resolveName`, `boing_resolveSlashAppeal`, `boing_reverifySlashAppealBallots`, `boing_reverseName`, `boing_rpcSupportedMethods`, `boing_simulateContractCall`, `boing_simulateTransaction`, `boing_submitIntent`, `boing_submitSlashAppeal`, `boing_submitTransaction`, `boing_verifyAccountProof`, `boing_verifyBlobDasSuccinctProof`, `boing_verifyBlsAggregate`, `boing_verifyBlsAggregateResolve`.

Implementation / single source of truth: `BOING_RPC_SUPPORTED_METHODS` in `crates/boing-node/src/rpc.rs`. A unit test (`spec_lists_every_supported_method`) fails the build if a method in that list is missing from this file, so add a section here whenever you add a method. For exact wire formats of transactions and signatures see BOING-SIGNED-TRANSACTION-ENCODING.md; for the machine-readable catalog call `boing_getRpcMethodCatalog` / `boing_getRpcOpenApi`.

boing_submitTransaction

Submit a signed transaction to the mempool.

Field	Type	Description
Params	`[hex_signed_tx]`	Hex-encoded bincode-serialized SignedTransaction

Example:

```
{"jsonrpc":"2.0","id":1,"method":"boing_submitTransaction","params":["0x..."]}
```

Result: `{ tx_hash: "0x<64 hex>", tx_id: "0x<64 hex>" }`: the transaction id (BLAKE3 of the unsigned tx; the key for `boing_getTransactionReceipt`). Nodes before 2026-10-16 returned `{ tx_hash: "ok" }`; clients should fall back to deriving the id from the signed bytes (SDK `submittedTxHash`). A duplicate submit (`-32000`, `data.reason = "already_in_mempool"`) also carries `data.tx_hash`.

Native value (testnet-v0.1.18+): a `ContractCallWithValue` payload (or a batch with a value step) submitted before the activation height (testnet 1150) is rejected with `-32000`, `data: { reason: "native_value_not_active", active_from }`. See NATIVE-VALUE-AND-WBOING.md.

P2P: When `--p2p-listen` is set and the tx is **admitted** to the mempool, the node also **gossips** the same `SignedTransaction` on libp2p topic `boing/transactions` so peers can verify and enqueue it locally (TECHNICAL-SPECIFICATION.md §12.3, RUNBOOK.md §8.1).

Admission order (node-side): signature verify (chain-id v2 + optional `priority_tip`) -> duplicate check -> sender must exist with a balance (`unknown_sender`) -> nonce must not be below the account nonce (`stale_nonce`; *future* nonces are admitted and wait in the per-sender queue) -> balance must cover the failed-tx fee (`insufficient_fee`) -> native-value / open-validator / name / media-blob activation checks -> fee affordability dry run -> insert. Mempool limits: 16 txs per sender, 10,000 total (`BOING_MEMPOOL_MAX_TXS`), 4 zero-fee txs per sender. Block building drains **nonce-contiguous runs per sender, highest effective tip first**.

Rejection shapes. All use JSON-RPC code `-32000` unless noted; branch on `error.data.reason`, not on the message text:

`data.reason`	Extra `data` fields	Meaning / client action
`already_in_mempool`	`tx_hash`	Exact same signed payload already pending; wait.
`unknown_sender`	-	Sender account has no on-chain balance; fund it (faucet/transfer) and retry.
`stale_nonce`	`expected_nonce`, `got_nonce`	A tx with this nonce already executed; re-read `nonce` via `boing_getAccount` and re-sign.
`insufficient_fee`	`have`, `need` (decimal strings, base units)	Balance below the protocol failed-tx fee (+ `priority_tip`).
`free_pending_limit`	`limit`	Legacy zero-fee path only.
`free_tx_balance_gate`	`have`, `need`	Legacy zero-fee path only.
`native_value_not_active`	`active_from`	Value call before activation height.
`pool_full` / `deployer_cap`	-	Codes `-32055` / `-32056` (QA pool).

Other codes: `-32602` malformed hex or undecodable transaction (clients must serialize with the same bincode layout; `priority_tip` is **omitted** from the encoding when zero, so decode with `SignedTransaction::from_bincode`, never raw `bincode::deserialize`), `-32050` QA rejected, `-32051` QA pending pool, `-32054` QA pool disabled, `-32016` rate limited.

QA pool (Unsure): If mempool QA returns Unsure **and** governance allows the pool to accept work (`qa_pool_config`: non-zero `max_pending_items` and `public_membership`, `administrators`, or `dev_open_voting`), the node enqueues the deployment and responds with **`-32051`** and `data: { "tx_hash": "0x..." }`. The same signed deploy is **gossiped** so peers can enqueue it. With **`public_membership`**, the node also admits the deploy for inclusion so apply can register it on-chain (no contract created until quorum Allow). Voters submit a signed **`QaPoolVote`** transaction (via `boing_submitTransaction` or as the 4th param to `boing_qaPoolVote`). Hard caps (`max_pending_items`, `max_pending_per_deployer`) prevent pool congestion; when full, **`-32055`** / **`-32056`** apply instead of enqueueing.

Operator RPC: **`boing_operatorApplyQaPolicy`**, **`boing_operatorApplyFeePolicy`**, **admin-only** **`boing_qaPoolVote`**, **`boing_registerBlsDualKey`**, **`boing_putBlobSampleOffer`** (node-local offer book writes), **`boing_submitSlashAppeal`**, and **`boing_resolveSlashAppeal`** require HTTP header **`X-Boing-Operator: <token>`** matching the node's **`BOING_OPERATOR_RPC_TOKEN`**. **Fail-closed:** if the variable is unset or empty, these methods are rejected with **`-32057`** (earlier releases treated unset as open). **`public_membership`** **QA votes do not use the operator header** - eligibility is protocol-side (signed vote tx; `-32053` if ineligible). **Breaking change for existing deployments:** nodes that rely on these methods must set **`BOING_OPERATOR_RPC_TOKEN`** and send **`X-Boing-Operator`**.

Does the pool need RPC to "run"? No. The node **owns** the pool: when QA returns Unsure and governance allows it, enqueueing happens inside normal transaction/mempool handling (`boing_submitTransaction` may return **`-32051`**). JSON-RPC is how clients *inspect and vote*—** `boing_qaPoolList` **, **`boing_qaPoolConfig`**, **`boing_qaPoolVote`**. Policy changes use **`boing_operatorApplyQaPolicy`**. Explorers and wallets are **read-and-submit clients only**; they must not scrape fees or pay voters. Voter rewards are credited in **apply** from `PROTOCOL_TREASURY`.

boing_listBlobSampleServeChallenges

Read-only list of SMT sample-serve challenges (Phase 10+). Phase 24 adds filters and optional list-all.

Field	Type	Description
Params	`[blobId]` or `[{ blobId?, status?, includeTerminal? }]`	`status`: `open` | `responded` | `failedServe`. `includeTerminal` defaults **true**. Omit `blobId` to list all.

Terminal rows (Responded / settled FailedServe) are pruned from consensus state after `deadline + 4096` via housekeeping; this RPC clones + display-expires Open past deadline only.

boing_getBlobSampleServeChallenge

Phase 24 get-by-key lookup for one sample-serve challenge.

Field	Type	Description
Params	`[{ blobId, chunkIndex, provider, challenger }]`	All fields required

Result: `{ tipHeight, challenge: { … } }` or **`null`** if missing.

boing_listSlashRecords

Read-only view of the consensus **`SlashAppealRegistry`** joined with the node-local mirror (equivocation / liveness / FailedServe slashes + appeals).

Field	Type	Description
Params	`[]` or `[{ reason?, slashId?, appealStatus?, includeExpired?, includeResolved? }]`	All filters optional. `appealStatus`: `pending` | `approved` | `rejected` | `expired`. `includeExpired` / `includeResolved` default **true** (back-compat); set **false** to hide Expired or Approved/Rejected consensus appeals.

Result:


```
{
  "slashes": [
    {
      "id": 1,
      "validator": "0x…",
      "amount": "5000",
      "reason": "equivocation",
      "block_height": 7,
      "appeal_deadline": 1007,
      "reversed": false
    }
  ],
  "appeals": [
    {
      "id": 1,
      "slash_id": 1,
      "status": "pending",
      "evidence_len": 12
    }
  ],
  "persisted": false
}
```

Notes: When the node has a **data directory**, the registry is written to `slash_registry.json` and `persisted` is `true`. Appeal submit/resolve: `boing_submitSlashAppeal` / `boing_resolveSlashAppeal` (operator auth: `X-Boing-Operator` matching `BOING_OPERATOR_RPC_TOKEN`; rejected with `-32057` if the token is unset). See TECHNICAL-SPECIFICATION.md §12.3. Consensus book also exposes `registryCommitment`. For ballot crypto audit use `boing_reverifySlashAppealBallots`. Phase 23: Expired appeals are pruned from the book after `resolve_deadline + 4096` via housekeeping. Phase 25: Approved/Rejected prune on the same TTL; `includeResolved` lets operators hide them while retained. Phase 26–32: `boing_getMediaBlobParams` echoes `blsAggregateScaffold` / `blsAggregateWired` / `blsPopRequired` / `blsAggregateSubmitLibraryWired` / `blsAggregateSubmitConsensusWired` / `blsCommitteeFlipHeight` / `blsCommitteeFlipActiveAtTip` / dynamic `blsLiveTx32StillEd25519` (Phase 32: opt-in flip of multi-pin verify from Ed25519 **32** → BLS aggregate **33** via `BOING_MEDIA_BLOBS_BLS_COMMITTEE_FLIP_HEIGHT`; unset = Phase 31 behavior). Dual-key host registry: `boing_listBlsDualKeys` / `boing_registerBlsDualKey` (`accountId`, `blsPubkey`, `blsPop`, optional `chainId`; operator auth required; rejected with `-32057` if `BOING_OPERATOR_RPC_TOKEN` is unset) / `boing_verifyBlsAggregate` / `boing_verifyBlsAggregateResolve`.

boing_reverifySlashAppealBallots

Phase 22/23 read-only audit of consensus slash-appeal resolve ballots (Ed25519 dual-verify V2 then V1), pin-set match, and registry commitment. **Never mutates** book state.

Field	Type	Description
Params	`[{ appealId?, slashId?, includeUnattested?, includeExpired?, includeResolved?, expectedRegistryCommitment? }]` or object	All filters optional; `includeUnattested` defaults **true**; `includeExpired` / `includeResolved` default **false** (skip Expired and Approved/Rejected)

Result (shape): `{ chainId, registryCommitment, registryCommitmentMatch?, appeals: [{ appealId, slashId, status, pinSetMatch, okCount, invalidCount, unattestedCount, allAttestedBallotsValid, votes: [{ voter, approved, weight, status: "ok"|"invalid"|"unattested", domain? }] }] }`.

boing_submitSlashAppeal

Submit an appeal against a recorded slash (within the appeal window).

Field	Type	Description
Params	`[slash_id, evidence_hex]` or `{ "slash_id", "evidence" }`	`slash_id` number; `evidence` hex (optional `0x`, may be empty)
Auth	Operator	Same as `boing_qaPoolVote`; requires `BOING_OPERATOR_RPC_TOKEN` to be set on the node (rejected with `-32057` otherwise)

Result: `{ "appeal_id": N, "slash_id": N }`

Errors: `-32060` with registry error message (not found, window closed, duplicate, …).

boing_resolveSlashAppeal

Approve or reject a pending appeal. Approval restores burned stake from the fee burn sink.

Field	Type	Description
Params	`[appeal_id, approved]` or `{ "appeal_id", "approved" }`	`approved` bool (or string `true` / `approve` / `1`)
Auth	Operator	Same as **`boing_qaPoolVote`**; requires **`BOING_OPERATOR_RPC_TOKEN`** to be set on the node (rejected with `-32057` otherwise)

Result: `{ "appeal_id": N, "approved": bool, "restored": "<u128 decimal>" }`

Errors: `-32061` with registry error message.

boing_qaPoolList

List pending items in the community QA pool (same `tx_hash` keys as `-32051`).

Field	Type	Description
Params	`[]`	None
Result	`{ items: [...] }`	Each item: `tx_hash`, `bytecode_hash`, `deployer` (hex), `allow_votes`, `reject_votes`, `age_secs`, and when present on the signed deploy: `purpose_category`, `asset_name`, `asset_symbol`.

boing_qaPoolVote

Cast a vote on a pending pool item.

Field	Type	Description
Params	`[tx_hash_hex, voter_hex, vote]` or `[tx_hash_hex, voter_hex, vote, signed_vote_tx_hex]`	`vote` is `allow`, `reject`, or `abstain` (case-insensitive). **`public_membership`:** any 32-byte account may vote, but the 4th param **must** be a hex-encoded signed `QaPoolVote` transaction (sender = `voter_hex`, payload subject/vote must match). **Admin-only** configs: listed `administrators` (or `dev_open_voting` with empty admins); operator header may apply. **`-32053`** if the voter is ineligible (not a member, deployer voting on their own item, or below `min_voter_stake`). **Anti-sybil:** one vote per account per item (later votes overwrite). **Abstain** does not count toward quorum and does **not** pay unless `pay_abstain` is true.
Result	`{ outcome: "pending" | "reject" | "allow", mempool?: boolean, duplicate?: boolean, error?: string }`	On public membership, Allow/Reject **resolution and voter payouts happen in apply** when the signed vote tx is included.

Errors: `-32052` no pending item; `-32053` voter ineligible; **`-32057`** operator authentication required for **admin-only** votes (missing or wrong `X-Boing-Operator`, or no `BOING_OPERATOR_RPC_TOKEN` configured on the node).

Voter payout (apply): When a signed `QaPoolVote` is included and that vote is counted toward a resolving quorum, each unpaid counted voter is credited `reward_per_counted_vote` BOING from `PROTOCOL_TREASURY` (pro-rata if the treasury is short). Explorers/wallets do not pay voters.

boing_getNetworkFeePolicy

Read the effective native fee policy (same JSON shape as `network_fee_config.json`).

Field	Type	Description
Params	`[]`	None
Result	object	`fee_validators_bps`, `fee_treasury_bps`, `fee_burn_bps` (must sum to 10000), `extra_fixed_fee` (u128 decimal string), `transfer_amount_bps`, `execution_gas_fee_bps`, `tip_proposer_bps` / `tip_treasury_bps` / `tip_burn_bps` (must sum to 10000; default 100%/0%/0), `treasury_account` (`0x5452454153555259…0001`), `fee_formula`, `micro_fee_execution_gas_bps`.

Recommended operator policy: **`execution_gas_fee_bps=100`** (micro; ~0.00021 BOING/transfer) with protocol gas **100% treasury** and tips **100% proposer**. Apply via **`npm run apply-micro-fee-policy`**. Block **emission** still goes to the round **proposer**. See OPTIONAL-PRIORITY-FEES.md.

boing_operatorApplyFeePolicy

Replace the node's native fee policy (persists `network_fee_config.json`). Requires **`X-Boing-Operator`** matching **`BOING_OPERATOR_RPC_TOKEN`**; rejected with `-32057` if the token is unset.

Field	Type	Description
Params	`[network_fee_config_json]`	One JSON string.
Result	`{ ok: true }`	Policy applied.

boing_operatorApplyQaPolicy

Replace the node's in-memory QA registry and pool governance config (same effect as loading `qa_registry.json` / `qa_pool_config.json` at startup, plus persistence to the node data directory). Intended for operators; requires **`X-Boing-Operator`** matching **`BOING_OPERATOR_RPC_TOKEN`**; rejected with `-32057` if the token is unset.

Field	Type	Description
Params	`[qa_registry_json, qa_pool_config_json]`	Two **strings**, each the full JSON document (not a path).
Result	`{ ok: true }`	Policy applied.

Errors: **`-32057`** missing or wrong operator header, or no `BOING_OPERATOR_RPC_TOKEN` configured on the node (fail-closed); **`-32602`** invalid JSON or schema.

CLI: `boing qa apply --registry <path> --pool <path> [--operator-token …]` (also reads `BOING_OPERATOR_RPC_TOKEN`).

boing_qaPoolConfig

Read effective QA pool governance parameters and current queue depth (no params).

Field	Type	Description
Params	`[]`	None
Result	object	`max_pending_items`, `max_pending_per_deployer`, `review_window_secs`, quorum/threshold fractions, `default_on_expiry`, `dev_open_voting`, `public_membership`, `min_voter_stake`, `min_quorum_votes`, `reward_per_counted_vote`, `pay_abstain`, `administrator_count`, `accepts_new_pending`, `pending_count`.

boing_getQaRegistry

Return the **effective protocol QA rule registry** the node uses for deployment checks (read-only, no authentication). Same JSON shape as on-disk `qa_registry.json` and as the first argument to `boing_operatorApplyQaPolicy`.

Field	Type	Description
Params	`[]`	None
Result	object	`max_bytecode_size`, `blocklist` (array of 32-byte arrays), `scam_patterns`, `always_review_categories` (array of strings), `content_blocklist`.

Reference: Canonical baseline JSON for comparison lives in the monorepo at `docs/config/qa_registry.canonical.json` — see `docs/config/CANONICAL-QA-REGISTRY.md`. Live nodes may differ after governance updates.

boing_clientVersion

Field	Type	Description
Params	`[]`	None

Result: string — e.g. `boing-node/0.1.0` (crate version). Lets clients and `npm run probe-rpc` confirm they are talking to a freshly built binary.

boing_rpcSupportedMethods

Field	Type	Description
Params	`[]`	None

Result: JSON array of strings — sorted list of `boing_*` method names implemented by this process (including `boing_clientVersion` and `boing_rpcSupportedMethods`). Intended for discovery and debugging; keep in sync with the node's RPC router when adding methods.

boing_chainHeight

Return the height of the latest **committed** block on this node (see Chain tip, committed height, and finality).

Field	Type	Description
Params	`[]`	None

Result: `u64`

boing_health

Lightweight **liveness** and **build identity** for probes, dashboards, and load balancers. Single round-trip; includes committed `head_height` and the same optional `chain_id` / `chain_name` semantics as `boing_getNetworkInfo` (from `BOING_CHAIN_ID` / `BOING_CHAIN_NAME` on the node process).

Field	Type	Description
Params	`[]`	None

Result:

```
{
  "ok": true,
  "client_version": "boing-node/0.1.0",
  "chain_id": 6913,
  "chain_name": "Boing Testnet",
  "head_height": 42,
  "storage_tree_v2_activation_height": null,
  "receipt_retention_blocks": 1000000,
  "tx_signing": {
    "v2_supported": true,
    "domain": "BOING_TX_V2",
    "chain_id": 6913,
    "v2_from_height": 900,
    "v2_required": false,
    "failed_tx_fee_from_height": 900,
    "block_signatures_from_height": 920,
    "registration_proof_domain": "BOING_OPEN_VALIDATOR_REGISTER_V2",
    "registration_proof_v2_from_height": 920,
    "priority_tips_supported": true,
    "priority_tip_domain": "BOING_TX_PRIORITY"
  },
  "client_hints": {
    "min_wallet_version": "0.1.18",
    "recommended_wallet_version": "0.1.22",
    "recommended_sdk": "0.5.38",
    "sdk_npm_package": "@boing-network/boing-sdk",
    "note": "Advisory only ..."
  },
  "rpc_surface": {
    "jsonrpc_batch_max": 32,
    "websocket_max_connections": 512,
    "http_rate_limit_requests_per_sec": 100,
    "ready_min_peers": null,
    "http_max_body_megabytes": 8,
    "get_logs_max_block_range": 128,
    "get_logs_max_results": 2048,
    "max_log_topic_filters": 4,
    "tx_payload_batch": true,
    "tx_batch_max_calls": 16,
    "tx_batch_max_calldata_bytes": 65536
  },
  "rpc_metrics": {
    "rate_limited_total": 0,
    "json_parse_errors_total": 0,
    "batch_too_large_total": 0,
    "method_not_found_total": 0,
    "websocket_cap_rejects_total": 0
  }
}
```


- **`chain_id`** / **`chain_name`**: **`null`** when the corresponding env var is unset or invalid (same as **`boing_getNetworkInfo`**).

- **`rpc_surface`:** Operator-facing snapshot of limits (same sources as env vars **`BOING_RPC_MAX_BATCH`**, **`BOING_RPC_WS_MAX_CONNECTIONS`**, **`BOING_RPC_MAX_BODY_MB`**, node **`RateLimitConfig.requests_per_sec`**, **`BOING_RPC_READY_MIN_PEERS`**, plus fixed **`boing_getLogs`** bounds). **`ready_min_peers`** is **`null`** when no minimum is configured.
- **`rpc_metrics`:** Cumulative counters since process start (best-effort diagnostics).
- **`client_hints`:** Advisory version hints, also returned by **`boing_getNetworkInfo`**. **`min_wallet_version`** is the oldest Boing Express release that signs correctly for the network and displays amounts correctly (chain-bound v2 signing domain: 0.1.15; chain-bound validator-registration proof: 0.1.16; height-aware 18-decimal BOING amounts at `native_currency.base_units_from_height`: 0.1.18); **`recommended_wallet_version`** and **`recommended_sdk`** are the latest tested releases of Boing Express and `@boing-network/boing-sdk`. Compare as semver and show an upgrade prompt; the node never enforces them. Operators can override with **`BOING_MIN_WALLET_VERSION`**, **`BOING_RECOMMENDED_WALLET_VERSION`** and **`BOING_RECOMMENDED_SDK_VERSION`** so hints follow a release without rebuilding the node. Added in an additive, backwards-compatible way: clients that ignore unknown fields are unaffected.
- **`storage_tree_v2_activation_height`** / **`receipt_retention_blocks`:** Additive protocol parameters, returned identically by **`boing_health`** and **`boing_getNetworkInfo`** (older nodes omit them; treat a missing key as unknown). Explorers and indexers should read them instead of hardcoding.
  - **`storage_tree_v2_activation_height`** - JSON integer or **`null`**. **`null`** = not configured on this node: contract-storage trees use legacy 64-bit key routing and all state roots are unchanged (the default). An integer **H** means blocks at height **>= H** compute contract-storage roots with full 256-bit key routing (STORAGE-TREE-V2-ACTIVATION.md). It comes from **`BOING_STORAGE_TREE_V2_ACTIVATION_HEIGHT`** / **`--storage-tree-v2-activation-height`**, is not persisted, and must be identical on every node of a network; it reflects this node's configuration, not on-chain consensus state. A client that recomputes roots uses 256-bit routing exactly when `height >= storage_tree_v2_activation_height`.
  - **`receipt_retention_blocks`** - JSON integer or **`null`**. The number of most recent blocks whose receipts this node serves from **`boing_getTransactionReceipt`**, **`includeReceipts`**, **`boing_getLogs`** and the account tx index (`BOING_RECEIPT_RETENTION_BLOCKS`, default **1000000**). **`null`** = unbounded (the env value **`0`**). For a transaction older than `head_height - receipt_retention_blocks + 1`, **`boing_getTransactionReceipt`** returns **`null`**; backfill from blocks instead. Pruning runs every 256 blocks, so the cut-off is approximate by up to that many blocks.
- **`tx_signing.priority_tips_supported`** / **`priority_tip_domain`:** Wallets can discover that a non-zero `priority_tip` is signed under the **`BOING_TX_PRIORITY`** domain (see Priority tips and block signatures).

boing_getSyncState

Structured view of the node's committed chain tip for clients that want explicit **head** vs **finalized** fields. Today both heights are identical.

Field	Type	Description
Params	`[]`	None

Result:

```
{
  "head_height": 42,
  "finalized_height": 42,
  "latest_block_hash": "0x..."
}
```

- **`head_height`:** Same value as `boing_chainHeight`.
- **`finalized_height`:** Same as `head_height` in the current implementation; may diverge in a future release if the node exposes pre-commit or optimistic data.
- **`latest_block_hash`:** BLAKE3 hash of the tip block's header (`Block::hash()`), 32-byte hex with `0x` prefix.

boing_getNetworkInfo

Single-call snapshot for dApps: chain metadata (from env when configured), tip state, timing, client build, consensus summary, **`chain_native`** aggregates, and **`rpc.not_available`** (see dApp network discovery).

Field	Type	Description
Params	`[]`	None

Result (shape):

```
{
  "chain_id": 6913,
  "chain_name": "Boing Testnet",
  "storage_tree_v2_activation_height": null,
  "receipt_retention_blocks": 1000000,
  "head_height": 42,
  "finalized_height": 42,
  "latest_block_hash": "0x...",
  "target_block_time_secs": 2,
  "client_version": "boing-node/0.1.0",
  "consensus": {
    "validator_count": 4,
    "model": "hotstuff_bft"
  },
  "client_hints": {
    "min_wallet_version": "0.1.18",
    "recommended_wallet_version": "0.1.22",
    "recommended_sdk": "0.5.38",
    "sdk_npm_package": "@boing-network/boing-sdk"
  },
  "native_currency": {
    "symbol": "BOING",
    "decimals": 18,
    "base_unit_decimals": 18,
    "base_units_per_boing": "1000000000000000000",
    "base_units_from_height": 1050,
    "gas_price_base_units": "1000000000000",
    "note": "…"
  },
  "chain_native": {
    "account_count": 42,
    "total_balance": "…",
    "total_stake": "…",
    "total_native_held": "…",
    "as_of_height": 42
  },
  "developer": {
    "repository_url": "https://github.com/Boing-Network/boing.network",
    "rpc_spec_url": "https://github.com/Boing-Network/boing.network/blob/main/docs/RPC-API-SPEC.md",
    "dapp_integration_doc_url": "https://github.com/Boing-Network/boing.network/blob/main/docs/BOING-DAPP-INTEG
    "sdk_npm_package": "@boing-network/boing-sdk",
    "websocket": {
      "path": "/ws",
      "handshake": { "type": "subscribe", "channel": "newHeads" },
      "event_types": ["newHead"]
    },
    "api_discovery_methods": [
      "boing_getRpcMethodCatalog",
      "boing_getRpcOpenApi",
      "boing_rpcSupportedMethods"
    ],
    "dex_discovery_methods": [
      "boing_listDexPools",
      "boing_listDexTokens",
      "boing_getDexToken"
    ],
    "token_metadata_methods": [
      "boing_getTokenMetadata",
```

```
      "boing_listTokens"
    ],
    "http": {
      "live_path": "/live",
      "ready_path": "/ready",
      "jsonrpc_post_path": "/",
      "response_header_rpc_version": "x-boing-rpc-version"
    }
  },
  "end_user": {
    "chain_display_name": "Boing Testnet",
    "explorer_url": null,
    "faucet_url": null,
    "canonical_native_cp_pool": "0x5b60d6f8514d1a8a40340fa354951d778b0821cd3ff7e534d0bea50f67d38976",
    "canonical_native_dex_factory": "0x7322ccec1ffdbf25d3f60384db0f9a38f8d6121233daccd48d4477d6036aff2a",
    "canonical_native_dex_multihop_swap_router": "0xd8504729900a2d6dc8b8afc7bf24a18dc08732d24a4d2e3555ea5b7f28f
    "canonical_native_dex_ledger_router_v2": "0xd39d7bb18e986a6e39a69521d9b6f1fe403ff757ee8d409b89f2bcccc4df798
    "canonical_native_dex_ledger_router_v3": "0x082a13685fbb69999f2d1378b2f704f3103ae6929b18a87510b513cb790c53c
    "canonical_native_amm_lp_vault": "0xac7a635931b192f5cff054e9994242f54fbda579a46eba21d9787600971ec90f",
    "canonical_native_lp_share_token": "0x2f460046eb700ff8c7eebe6f4c409465a9111f32210edcc478a2a0de9d0ec2db"
  },
  "rpc": {
    "not_available": ["staking_apy"],
    "not_available_note": "…"
  }
}
```


- **`developer.dex_discovery_methods`:** Stable list of L1 native DEX discovery JSON-RPC method names (same triple as `docs/HANDOFF_Boing_Network_Global_Token_Discovery.md`). Clients may merge this with `boing_rpcSupportedMethods` / `boing_getRpcMethodCatalog` for capability detection.
- **`end_user`:** Wallet-facing strings and optional **32-byte hex** hints when operators set the matching **`BOING_CANONICAL_NATIVE_*`** env vars on the node: **`canonical_native_cp_pool`**, **`canonical_native_dex_factory`**, **`canonical_native_dex_multihop_swap_router`**, **`canonical_native_dex_ledger_router_v2`**, **`canonical_native_dex_ledger_router_v3`**, **`canonical_native_amm_lp_vault`**, **`canonical_native_lp_share_token`**. Omitted keys are JSON **`null`**. **`boing-sdk`** **`mergeNativeDexIntegrationDefaults`** merges these with app overrides; historical embedded **6913** constants apply **only** when network info is omitted (offline). A live snapshot with null canonical fields means the hosted chain has not published those contracts (BOING-DAPP-INTEGRATION.md, dApp network discovery).
- **`storage_tree_v2_activation_height`** / **`receipt_retention_blocks`:** Same fields and semantics as on **`boing_health`** (see the bullet there).
- **`chain_id`** / **`chain_name`:** From **`BOING_CHAIN_ID`** / **`BOING_CHAIN_NAME`** when set on the node process; **`chain_id`** is **`null`** when not configured.
- **`validator_count`:** Size of this node's in-process validator set (HotStuff BFT); not a network-wide discovery endpoint for all operators.

- **`native_currency`:** Native BOING has **18 decimals** (1 BOING = 10^18 base units; NATIVE-DENOMINATION.md). **`decimals`** is the unit of the amounts this node returns *now*: **18** (base units) on chains without history and on testnet from **`base_units_from_height`** (1050), **0** (whole BOING) on testnet before it. **`base_units_from_height`** is **`null`** when the chain is in base units from genesis. Blocks below **`base_units_from_height`** carry whole-BOING amounts; format historical blocks with the unit of their height. **`gas_price_base_units`**: fee per gas in base units from the switch. Same object in **`boing_health`**.
- **`chain_native`:** Sums over this node's committed account table; see dApp network discovery.
- **`developer`:** Stable links and discovery hints for integrators. URLs default to the public GitHub tree; operators may override with environment variables (see table below).

Variable	Effect
`BOING_DEVELOPER_REPOSITORY_URL`	Base repository URL used to derive default doc links when spec/dapp URLs are unset.
`BOING_DEVELOPER_RPC_SPEC_URL`	Overrides **`developer.rpc_spec_url`**.
`BOING_DEVELOPER_DAPP_DOC_URL`	Overrides **`developer.dapp_integration_doc_url`**.

HTTP probes `GET /live` and `GET /ready`

Plain-text endpoints for orchestrators (Kubernetes **`livenessProbe`** / **`readinessProbe`**, Docker **`HEALTHCHECK`**, load balancers) without parsing JSON-RPC.

Path	Meaning
`GET /live`	Process is running and the HTTP server is accepting connections. Does **not** read chain state.
`GET /ready`	Node can acquire a read lock on in-memory state (RPC is not wedged on the state mutex). Optionally also requires a minimum **P2P peer count** when **`BOING_RPC_READY_MIN_PEERS`** is set (see below).

Successful responses use HTTP **200** with a short body (`ok` / `ready`). If the peer requirement is not met, **`GET /ready`** returns **503** with a short `not_ready: peers …` message. They receive the same **`x-boing-rpc-version`** response header as **`POST /`**.

Request body limit (JSON-RPC `POST /`)

Large signed-transaction hex payloads (e.g. contract deploy) need a generous HTTP body limit. Default maximum request body size is **8 MiB**.

Variable	Effect
`BOING_RPC_MAX_BODY_MB`	Maximum JSON-RPC POST body size in **mebibytes** (integer, **≥ 1**). Invalid or missing values fall back to **8**.
`BOING_RPC_MAX_BATCH`	Max objects per JSON-RPC **batch** array (default **32**, hard cap **256**). `0` is invalid and falls back to the default.
`BOING_RPC_READY_MIN_PEERS`	When set to a positive integer, `GET /ready` returns **503** unless the node reports at least that many connected P2P peers. Unset or `0`: no peer check (read lock only).
`BOING_RPC_WS_MAX_CONNECTIONS`	Max concurrent `GET /ws` WebSocket connections (default **512** when unset; explicit **0** = unlimited). When exceeded, new handshakes receive **503**.

WebSocket `GET /ws` — newHeads

Browser-friendly push when the **local** committed tip changes (block produced or imported). Uses the same CORS allowlist as `POST /` plus `BOING_RPC_CORS_EXTRA_ORIGINS`.

1. Connect with `GET /ws` (WebSocket upgrade).
2. Send **one** text frame: `{"type":"subscribe","channel":"newHeads"}`.
3. Server replies `{"type":"subscribed","channel":"newHeads"}`.
4. On each committed tip update, server sends `{"type":"newHead","height":<u64>,"hash":"0x..."}` (BLAKE3 header hash, 32-byte hex).

Laggy subscribers may drop intermediate heads (**broadcast** channel); clients should reconcile with `boing_getSyncState` / `boing_chainHeight` if they need every height.

The default cap (**512**) keeps `newHeads` subscribers from exhausting file descriptors or CPU; public RPC operators can tune it with `BOING_RPC_WS_MAX_CONNECTIONS`. The testnet faucet also has a node-wide budget (`BOING_FAUCET_MAX_PER_MINUTE`, default **30**, **0** = unlimited) on top of the per-address cooldown.

boing_getRpcMethodCatalog

Returns the embedded **method catalog** (JSON Schema–style `params` / `result` hints per method) shipped with the node binary. Intended for codegen, docs generators, and IDE tooling.

Field	Type	Description
Params	`[]`	None

Result: JSON object with `description` and `methods` (array of `{ name, summary, params, result }`).

boing_getRpcOpenApi

Returns a **minimal OpenAPI 3.1** JSON document describing `POST /` (JSON-RPC envelope), `GET /ws` (subscribe protocol), and plain-text probes `GET /live` / `GET /ready`. Complements the human-readable spec and `boing_getRpcMethodCatalog`.

Field	Type	Description
Params	`[]`	None

Result: OpenAPI root object (`openapi`, `info`, `paths`, `components`, …).

boing_getBalance

Get the spendable balance for an account. **Recommended for wallets** (e.g. boing.express) to display balance without deriving from state.

Field	Type	Description
Params	`[hex_account_id]`	32-byte AccountId (hex)
Result	`{ balance: string, balance_boing: string, decimals: number }`	`balance`: native balance in **ledger units** (u128 decimal string): base units, 1 BOING = 10^18, on chains without history and on testnet from height 1050; whole BOING before it. `decimals` (18 or 0) says which; `balance_boing` is the same amount formatted as BOING (e.g. `"20.021"`). Nodes before testnet-v0.1.17 return only `balance` (whole BOING). See NATIVE-DENOMINATION.md.

Example: `{"jsonrpc":"2.0","id":1,"method":"boing_getBalance","params":["0x..."]}` → `{"jsonrpc":"2.0","id":1,"result":{"balance":"1500000000000000000","balance_boing":"1.5","decimals":18}}`

boing_getAccount

Get full account state (balance, nonce, stake). **Recommended for wallets** to build transactions (nonce) and show balance/stake.

Field	Type	Description
Params	`[hex_account_id]`	32-byte AccountId (hex)
Result	`{ balance: string, nonce: number, stake: string, pending_unbond: string, unbond_unlock_height: number, balance_boing, stake_boing, pending_unbond_boing: string, decimals: number }`	balance, stake, and pending_unbond are u128 decimal strings in ledger units (base units from the 18-decimals switch; `decimals` 18 or 0); the `*_boing` fields are the same amounts formatted as BOING; nonce and unbond_unlock_height are u64. If account does not exist, returns zeros. `pending_unbond` is stake waiting for the unbonding delay; claim with `ClaimUnbond` after `unbond_unlock_height`.

Example: `{"jsonrpc":"2.0","id":1,"method":"boing_getAccount","params":["0x..."]}` → `{"jsonrpc":"2.0","id":1,"result":{"balance":"1000000","nonce":5,"stake":"0","pending_unbond":"0","unbond_unlock_height":0}}`

boing_getBlockByHeight

Get a block by height.

Field	Type	Description
Params	`[height]` or `[height, include_receipts]` or `[height, include_receipts, include_code]`	Block height (u64); optional booleans — `include_receipts` (default `false`) adds `receipts`; `include_code` (default `true`) controls large byte fields (see below).

Result: Block object or `null` if not found. `header` includes `receipts_root` (hex-encoded 32-byte Merkle root over execution receipts; see protocol spec). Optional `hash` field (hex) is added for convenience. With `include_receipts: true`, each entry matches **`boing_getTransactionReceipt`** shape or `null`.

Byte fields (bytecode, calldata, blob `data`, evidence, …): emitted as **`0x` + hex** (same style as receipt `return_data` / logs), **not** as JSON number arrays. Account ids / 32-byte hashes in the block JSON are also `0x` hex. When **`include_code: false`**, those heavy vectors are replaced with `{ "omitted": true, "size": <bytes>, "hash": "0x…" }` (BLAKE3 of the omitted bytes) so explorers can sample tip windows without multi-MiB payloads. Use **`boing_getContractCode`** when the full deployed bytecode is needed.

Off-chain deploy registry: indexers that need a **chain-wide list of predicted `ContractDeploy*` contract addresses** (and BLAKE3 **transaction ids**) can call this method with **`include_receipts: false`** and **`include_code: true`** (default) and parse **`transactions`**. The TypeScript SDK exposes **`extractUniversalContractDeploymentsFromBlock`** and related helpers (`boing-sdk`, **`universalContractDeployIndex.ts`**). An optional Cloudflare Worker materializes rows into D1 and serves HTTP/SSE; see HANDOFF_Universal_Contract_Deploy_Indexer.md.

boing_getTransactionReceipt

Lookup execution outcome for an included transaction (same id as `Transaction::id()` / mempool `tx_hash`).

Field	Type	Description
Params	`[hex_tx_id]`	32-byte transaction id (hex)

Result: Receipt object or `null` if unknown (e.g. not yet included, or node predates receipt persistence).

Receipt: `{ tx_id, block_height, tx_index, success, gas_used, return_data, logs, error }` — `return_data` is hex; `logs` is an array of `{ topics: string[], data: string }` (each topic is `0x` + 32-byte hex, `data` is hex). Logs are populated for `ContractCall` and for **`ContractDeploy` only when bytecode uses init-code mode** (leading `0xFD` prefix; see `TECHNICAL-SPECIFICATION.md` §4.4). `error` is set when `success` is false.

Failed transactions (security review #3; testnet from height 900, mainnet from genesis): a tx that passes admission (signature, nonce, fee balance) but fails in execution is still included. Its state changes are reverted, the sender's nonce advances, and the sender pays the failed-tx fee (`fee_for_gas(21000)` + any fixed fee, capped at the balance; 1 BOING today). The receipt has `success: false`, `error`, and `gas_used: 21000`. A failing Native Batch reverts every step and pays the fee once. Below the activation height a failed tx paid nothing (`gas_used: 0`).

boing_getLogs

Query execution logs across a **bounded** range of committed blocks. Intended for indexers and wallets; nodes enforce limits to keep scans cheap.

Field	Type	Description
Params	`[filter]`	Single object (camelCase keys below).
`fromBlock`	number or string	Start height (inclusive). JSON number or decimal / `0x` hex string.
`toBlock`	number or string	End height (inclusive). Must be `>= fromBlock`.
`address`	string (optional)	32-byte `AccountId` hex. When set, only logs attributed to that contract are returned (`ContractCall` target, or the deployed contract address for deploy txs that used **init-code** mode — leading bytecode byte `0xFD`, see `TECHNICAL-SPECIFICATION.md` §4.4).
`topics`	array (optional)	Up to 4 entries; each is `null` (wildcard) or a 32-byte topic hex string. **Positional match:** entry `i` filters `topic_i` of each log (same index slot).

Limits (reference implementation): inclusive block span at most **128**; at most **2048** log entries per call. Exceeding span returns JSON-RPC error `-32602`; exceeding the result cap returns `-32603`.

Result: JSON array of objects:

```
{ block_height, tx_index, tx_id, log_index, address, topics, data }
```

- `address` is hex or `null` when the node cannot attribute an emitting contract.
- Order is block height ascending, then transaction order in the block, then `log_index`.

Example: `{"jsonrpc":"2.0","id":1,"method":"boing_getLogs","params":[{"fromBlock":1,"toBlock":10,"topics":[null,"0x000…"]}]}`

boing_getBlockByHash

Get a block by hash.

Field	Type	Description
Params	`[hex_block_hash]` or `[hex_block_hash, include_receipts]` or `[hex_block_hash, include_receipts, include_code]`	Optional booleans — same as `boing_getBlockByHeight`.

Result: Block object or `null` if not found.

boing_getAccountProof

Get a Merkle proof for an account.

Field	Type	Description
Params	`[hex_account_id]`	32-byte AccountId (hex)

Result: `{ proof: string, root: string, value_hash: string }`

boing_verifyAccountProof

Verify an account Merkle proof.

Field	Type	Description
Params	`[hex_proof, hex_state_root]`	Proof and expected root (hex)

Result: `{ valid: boolean }`

boing_getContractCode

Read-only. Returns the deployed bytecode of a contract account and its code hash.

Param	Type	Description
`hex_contract_id`	string	32-byte account id (`0x` + 64 hex).

Result: `{ "bytecode": "0x…", "code_hash": "0x…", "size": <bytes> }`, or **`null`** when the account has no bytecode (EOA or unknown account). `code_hash` is BLAKE3(bytecode), the same `code_hash` folded into the account state leaf. Invalid params → `-32602`.

SDK: `client.getContractCode(hexContractId)` → `ContractCode | null`.

boing_getContractStorage

Read a single 32-byte Boing VM storage word for a contract (same semantics as `SLOAD`: missing slot → zero word). Useful for indexers and wallets that know the storage key layout (e.g. reference NFT / token conventions — BOING-REFERENCE-NFT.md XOR owner/metadata keys; a **`mint_batch`** still writes those slots under one receipt). This helper is unrelated to JSON-RPC **HTTP** batching (`BOING_RPC_MAX_BATCH`).

Field	Type	Description
Params	`[hex_contract_id, hex_storage_key]`	Two 32-byte values (hex)

Result: `{ value: string }` — `value` is `0x` + 64 hex chars (32 bytes).

boing_listDexPools / boing_listDexTokens / boing_getDexToken

First-class **DEX-derived discovery** (see HANDOFF_Boing_Network_Global_Token_Discovery.md): pools and tokens come from the on-chain pair directory plus live pool storage reads — **not** “every ledger account.”

Field	Type	Description
Env	—	`BOING_CANONICAL_NATIVE_DEX_FACTORY` (same id as `boing_getNetworkInfo.end_user.canonical_native_dex_factory`) when `params.factory` is omitted. If neither is set, these methods return `-32000`. Optional `BOING_DEX_TOKEN_METADATA_SCAN_BLOCKS` (default **8192**, max **500000**): how many recent blocks the node scans for `ContractDeployWithPurposeAndMetadata` (`asset_symbol` / `asset_name`) when enriching token rows. Optional `BOING_DEX_DISCOVERY_MAX_RECEIPT_SCANS`: max receipt rows examined per call for `Log3` register matching (default **500000**; set `0` for unlimited). Optional `BOING_DEX_TOKEN_DECIMALS_JSON`: JSON object `{ "0x" + 64hex: number }` setting `decimals` for `boing_listDexPools` (`tokenADecimals` / `tokenBDecimals`), `boing_listDexTokens`, and `boing_getDexToken`. Tokens not in the map whose deployed code is a reference fungible template (v2, v1, secured; all whole units) report `0`; anything else reports `null` (unknown; nodes before 2026-10-17 reported a guessed **18**).
Params	`[object]`	Single JSON object (or `[object]` one-element array). Common fields: `factory` (optional **32-byte hex** directory id — overrides env for multi-directory / self-serve RPC), `cursor`, `limit`. `light` **: true** (or `enrich` **: false**) skips receipt + deploy scans: `createdAtHeight` / `firstSeenHeight` are JSON `null`, and token `metadataSource` is `abbrev` only. Token listing also accepts `minReserveProduct` and `minLiquidityWei` as **decimal digit strings** (unsigned integers). `includeDiagnostics` **: true** adds a top-level `diagnostics` object (pool and token responses) or embeds it on `boing_getDexToken` when present.

Persistence: When the node is started with a **data directory** and receipt persistence enabled, committed `ExecutionReceipt` blobs (`receipts_{height}.bin`) for heights `0..=tip` are loaded into the in-memory receipt map used by these methods, so `createdAtHeight` / `firstSeenHeight` reflect every receipt the process has loaded (same trust model as `boing_getTransactionReceipt` for those heights).

`boing_listDexPools` **result:** `{ pools: [...], nextCursor: string | null }` — each pool has `poolHex`, `tokenAHex`, `tokenBHex`, `tokenADecimals` / `tokenBDecimals` (same resolution as token-list `decimals`: `BOING_DEX_TOKEN_DECIMALS_JSON`, else **0** for reference fungible template code, else `null`), `feeBps`, `reserveA` / `reserveB` (decimal strings), and `createdAtHeight`: committed block height of a validated factory `Log3` register for that pool when known (else JSON `null`, e.g. `light` mode). Optional `diagnostics` when `includeDiagnostics` was set.

`boing_listDexTokens` **result:** `{ tokens: [...], nextCursor: string | null }` — each token has `id`, `symbol`, `name`, `decimals` (integer, or `null` when unknown), `decimalsSource` (`env` = operator map, `template` = reference fungible template code, whole units; `null` when unknown), `poolCount`, `firstSeenHeight` (minimum known pool registration height for that token, else `null`), and `metadataSource`: `deploy` when `symbol` **/** `name` were taken from a recent metadata deploy, else `abbrev`. `nextCursor` is the last token's `id` when another page may exist; otherwise `null`. Optional `diagnostics` when `includeDiagnostics` was set.

`boing_getDexToken` **params:** `{ "id": "0x" + 64 hex, "factory"?: ..., "light"?: bool, "includeDiagnostics"?: bool }` — **result** is one token object or JSON `null`. `diagnostics` may appear on the token object when requested.

Pagination cursors: pool pages use opaque strings of the form `i{start_index}` (implementation detail; treat as opaque). Token pages use the last returned `id` as the next `cursor`.

SDK: `BoingClient.listDexPoolsPage`, `listDexTokensPage`, `getDexToken`.

Regression: `cargo test -p boing-node --test native_dex_factory_rpc_happy_path`.

boing_getTokenMetadata / boing_listTokens

Token name, symbol, decimals and total supply for **any** token contract, **not only tokens that sit in a DEX pool** (contrast `boing_getDexToken`). **Read-only and non-consensus**; no DEX factory is needed. The node builds an in-memory index from the blocks and receipts it already holds (incrementally, re-anchored on a reorg).

`boing_getTokenMetadata` **params**	`["0x" + 64 hex]`, `[{ "id": "0x…" }]` or `{ "id": "0x…" }`. A malformed id returns `-32602`.
`boing_getTokenMetadata` **result**	Token object, or JSON `null` when the id has no contract code and no recorded deploy.
`boing_listTokens` **params**	`[{ "limit"?: 1–500 (default 100), "cursor"?: last id from` nextCursor `}]`.
`boing_listTokens` **result**	`{ tokens: [...], nextCursor: string | null, total, indexedThroughHeight }`, ordered by `id`. Directory = every contract that declared a name/symbol at deploy **or** runs a known token implementation (reference fungible v1/v2/secured, wrapped BOING).

Token object: `id`, `name`, `symbol`, `decimals`, `decimalsSource`, `totalSupply`, `totalSupplySource`, `standard`, `metadataSource`, `deployHeight`, `deployer`, `deployTx`. Every field except `id` may be `null`; the node reports only what exists on chain and never guesses.

Field	Where it comes from
`name` / `symbol`	The **deploy transaction only** (`ContractDeployWithPurposeAndMetadata` : `asset_name` ≤ 256 bytes, `asset_symbol` ≤ 32). Contract storage never holds them. A token deployed without that metadata has **no** name or symbol anywhere on chain, so both are **`null`** and `metadataSource` is **`null`** (otherwise **`"deploy"`**). No abbreviated placeholder is ever returned.
`decimals` / `decimalsSource`	Operator map `BOING_DEX_TOKEN_DECIMALS_JSON` (`"env"`), else **`0`** for reference fungible template code (`"template"` : those contracts store whole-unit balances and have no decimals slot), else **`18`** for wrapped BOING, else **`null`** (unknown). Same resolver as `boing_getDexToken` .
`totalSupply` / `totalSupplySource`	Decimal string (u128). Wrapped BOING: read from storage (`"storage"`). Reference tokens have no supply slot; their supply is the amount of the single successful one-time `mint_first` call, found in the blocks and receipts the node still holds (`"mint_first"`). Otherwise **`null`** .
`standard`	`reference-fungible-v1` , `reference-fungible-v2` , `reference-fungible-secured-v1` , `reference-fungible-secured-v2` , `wrapped-boing` , or **`null`** (code not recognised).

Limits: the index covers the blocks the node holds, so a node that pruned history cannot report deploy metadata for old tokens. Tokens with unrecognised bytecode and no deploy metadata are not listed by `boing_listTokens` (they still resolve through `boing_getTokenMetadata` with mostly `null` fields). Older nodes answer **method not found** (`-32601`); clients should fall back to `boing_getDexToken` / `boing_listDexTokens` .

Public gateway class: `boing_getTokenMetadata` is **read**; `boing_listTokens` is **heavy_read** (`ops/cloudflare/public-rpc/policy.json`).

SDK: `BoingClient.getTokenMetadata` , `listTokensPage` . **Regression:** `cargo test -p boing-node token_metadata` and `cargo test -p boing-node --test token_metadata_rpc` .

Native constant-product AMM (chain 6913 — integration note)

Wallets and dApps still **configure** the pool **`AccountId`** in their own env (the JSON-RPC methods are generic). For **public testnet**, operators maintain a single **canonical** pool id so tutorials and UIs can default to one known-good deployment.

Canonical public testnet pool `AccountId`	**Hosted Fly RPC currently unset** (`end_user.canonical_native_cp_pool` is **`null`** as of 2026-08-24). Historical tunnel-ledger id **`0x5b60d6f8514d1a8a40340fa354951d778b0821cd3ff7e534d0bea50f67d38976`** (operator-published **2026-05-21**, documented deployer **`0x8079f30559c02c5fe087c9ad1bc3b751a29de35992f3f8431734243e825f696e`**) is **not** live on this cluster. Read **`boing_getNetworkInfo.end_user`** . After bootstrap, follow OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md. **CREATE2 note:** deployer **`0xc063512f…`** + current-tree bytecode predicts **`0xce4f8193…`** — not the live pool; run **`cargo run -p boing-execution --example verify_canonical_cp_pool_create2_drift`** .
boing.finance (separate repo)	**Source constant:** `frontend/src/config/boingCanonicalTestnetPool.js` → **`CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX`** . **Cloudflare Pages / CI build:** **`REACT_APP_BOING_NATIVE_AMM_POOL`** (same 64-hex `AccountId`). Also wire **`nativeConstantProductPool`** (or equivalent in `contracts.js`) for chain **6913**. Redeploy the app after any change to this hex. Optional: depend on npm **`boing-sdk`** and import **`CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX`** (mirror of this table — see row below) plus shared RPC helpers / `explainBoingRpcError` .
boing-sdk	**`CANONICAL_BOING_TESTNET_NATIVE_CP_POOL_HEX`** — exported from `boing-sdk/src/canonicalTestnet.ts` ; convenience for tutorials and TS apps. **Authoritative** values remain **this spec** and TESTNET.md §5.3.

Need	RPC / approach
Reserve A / B	Two **`boing_getContractStorage`** calls: `hex_contract_id` = pool account; storage keys are the **32-byte big-endian layout** for the pool program (see NATIVE-AMM-CALLDATA.md and `boing_execution::reserve_a_key` / `reserve_b_key`). Each `value` word holds the u128 reserve in the **low 16 bytes** (high 16 zero), matching reference-token word style.
Swap / liquidity	Signed **`contract_call`** transactions (calldata per NATIVE-AMM-CALLDATA.md); preflight **`boing_simulateTransaction`** / **`boing_qaCheck`** on deploy bytecode as usual. **Canonical pool deploy:** prefer **`create2_salt: Some(NATIVE_CP_POOL_CREATE2_SALT_V1)`** on purpose deploy (fixed salt in `native_amm.rs`) so the pool **`AccountId`** is predictable from deployer + bytecode — see NATIVE-AMM-CALLDATA.md § CREATE2.
Automated regression	Repository test: `cargo test -p boing-node --test native_amm_rpc_happy_path` (deploy → add liquidity → swap; asserts reserves via `boing_getContractStorage`).

Publish checklist (for **future** pool rotations): OPS-CANONICAL-TESTNET-NATIVE-AMM-POOL.md. Keep **boing.finance** and **`boing-sdk`** mirrors in sync with live **`end_user`** when a canonical pool is published.

Native DEX directory + ledger router (integration note)

Multi-pair flows use **generic** JSON-RPC (`contract_call` , `getContractStorage` , `getLogs` , `simulateTransaction`). Nodes with **`BOING_CANONICAL_NATIVE_DEX_FACTORY`** also expose **`boing_listDexPools`** / **`boing_listDexTokens`** / **`boing_getDexToken`** for cursor-paginated discovery (see

above). Specs and bytecode live in the node repo:

Artifact	Doc	Notes
Pair directory	NATIVE-DEX-FACTORY.md	`register_pair` / `pairs_count` / `get_pair_at`; count key + `Log3` indexing
Ledger router	NATIVE-DEX-LEDGER-ROUTER.md	Forwards **128-byte** pool calldata via nested `Call` — **v1/v3 pools only**
SDK	`boing-sdk`	`nativeDexFactory*`, `nativeDexLedgerRouter`, `create2` prediction helpers
Regression	`cargo test -p boing-node --test native_dex_factory_rpc_happy_path`	Deploy pool + directory → `register_pair` → `pairs_count` / `get_pair_at` receipts

Materialized pool directory (HTTP, not JSON-RPC): this monorepo ships a Cloudflare Worker (`workers/native-dex-indexer/`) that mirrors the indexer's `pools[]` into **D1** and exposes **`GET /v1/directory/meta`** and cursor-paginated **`GET /v1/directory/pools`**. Use **`boing-sdk`** **`fetchNativeDexDirectoryMeta`**, **`fetchNativeDexDirectoryPoolsPage`**, **`collectAllNativeDexDirectoryPools`** (`nativeDexDirectoryApi.ts`). Ops, secrets, R2, and protocol follow-ups: HANDOFF_NATIVE_DEX_DIRECTORY_R2_AND_CHAIN.md. Local smoke: **`npm run verify-native-dex-directory-worker`** from the repo root.

boing_simulateContractCall

Dry-run a **`contract_call`** without building or signing a full **`SignedTransaction`**. Execution uses the same VM path as **`boing_simulateTransaction`** on a **state snapshot** (no commit to committed chain).

Field	Type	Description
Params	`[contract_hex, calldata_hex, sender_hex?, at_block?]`	**`contract_hex`**: 32-byte `AccountId` (`0x` + 64 hex). **`calldata_hex`**: VM calldata (hex). Max **256 KiB** decoded length. **`sender_hex`**: optional; JSON **`null`** or omit → synthetic **all-zero** `AccountId` (ephemeral account with nonce **0** inserted in the snapshot if missing). Otherwise **`sender`** **must exist** in committed state; **nonce** is taken from that account's committed nonce. **`at_block`**: optional **`"latest"`**, **`null`**, or integer **equal to current committed tip height**; other heights return **`-32602`** (historical replay not implemented).
Result	object	Same shape as **`boing_simulateTransaction`** **`result`**: **`gas_used`**, **`success`**, **`return_data`**, **`logs`**, **`error`** (when `success` is false), **`suggested_access_list`**, **`access_list_covers_suggestion`**. The synthetic tx uses **`access_list`** = heuristic suggestion for **`ContractCall`**, so **`access_list_covers_suggestion`** is **`true`**.

Errors: `-32602` for invalid params (malformed hex, calldata too long, unknown `at_block`, missing sender account when non-zero sender). VM execution failures return `success: false` in `result` (same as `boing_simulateTransaction`), not a JSON-RPC error.

Tests: `cargo test -p boing-node --test simulate_contract_call_rpc`.

Extended rationale and ecosystem rollout: PROTOCOL_NATIVE_DEX_RPC_AND_INDEXING_ROADMAP.md §1.

boing_simulateTransaction

Simulate a transaction without applying it.

Field	Type	Description
Params	`[hex_signed_tx]`	Hex-encoded SignedTransaction

Result: `{ gas_used: number, success: boolean, return_data: string, logs?: array, error?: string, suggested_access_list: { read: string[], write: string[] }, access_list_covers_suggestion: boolean, network_fee?: string, network_fee_decimals?: number }` — `network_fee` (success only, testnet-v0.1.17+) is the fee this tx would pay if included in the next block, in ledger units (`network_fee_decimals` 18 = base units); on success, `return_data` is hex (contract return buffer) and `logs` matches receipt log shape; on failure, `return_data` is `"0x"` and `logs` is `[]`. `suggested_access_list` is a **heuristic** minimum account set for parallel scheduling from the tx payload (see `TECHNICAL-SPECIFICATION.md` §4.2); `access_list_covers_suggestion` is `true` when the signed tx's declared access list includes every account in that suggestion (read or write). Hints are included on both success and failure so clients can fix access lists before submit.

From testnet-v0.1.18 the simulation runs with the rules of the **next** block (height tip + 1), so a value call can be previewed at the block before the native-value activation height (1150). A `ContractCallWithValue` simulation moves the value from the sender to the contract in the snapshot (the sender must hold `value` plus the fee).

For **unsigned** calldata-only simulation, use `boing_simulateContractCall` above.

boing_registerDappMetrics

Register a dApp for incentive tracking.

Field	Type	Description
Params	`[hex_contract, hex_owner]`	Contract and owner AccountIds (hex)

Result: `{ registered: true, contract: string, owner: string }`

boing_submitIntent

Submit a signed intent for solver fulfillment.

Field	Type	Description
Params	`[hex_signed_intent]`	Hex-encoded SignedIntent

Result: `{ intent_id: string }`

boing_qaCheck (optional — when QA is enabled)

Pre-flight check for a deployment without submitting. Allows clients to see whether bytecode (and optional purpose declaration) would be **Allow**, **Reject**, or **Unsure** (pool) before calling `boing_submitTransaction`.

Field	Type	Description
Params	`[hex_bytecode]` or `[hex_bytecode, purpose_category, description_hash?, asset_name?, asset_symbol?]`	Bytecode only runs size/opcode/blocklist rules. With `purpose_category`, the same **full mempool QA** path applies as in `boing_submitTransaction` for deploy payloads (including optional 32-byte `description_hash`, then optional `asset_name` / `asset_symbol` for content policy). To pass only name/symbol without a real description commitment, use a placeholder 32-byte hex (e.g. all zeros).
Result	`{ result, rule_id?, message?, doc_url? }`	`result`: `"allow"`, `"reject"` (rule_id/message when applicable), or `"unsure"` (community QA pool). Mirrors mempool `boing_qa` for contract deploy.

Errors: When QA is not enabled, returns `-32601` (method not found) or a dedicated code. When QA rejects: use structured error code below.

boing_faucetRequest (testnet only)

Request testnet BOING for an account. Only available when the node is started with `--faucet-enable`. **Do not enable on mainnet.**

Field	Type	Description
Params	`[hex_account_id]`	32-byte account ID (hex). Recipient of the faucet transfer.

Result: `{ ok: true, amount: number, to: string, message: string, tip_height: number, connected_peers: number, warning?: string }`

- **`tip_height` / `connected_peers`:** Chain tip and P2P peer count at accept time.

- **`warning`:** Present when tip is **0** (especially with **no peers**) — the transfer is only in the **local mempool** and may **never commit** until the node syncs or a validator produces blocks. Clients MUST NOT treat `ok: true` alone as "balance credited."

Rate limit: 1 request per 60 seconds per account ID. Returns `-32016` with message "Faucet cooldown" if called too soon.

P2P: Same as **`boing_submitTransaction`** — when **`--p2p-listen`** is set and the built transfer is admitted, the signed faucet tx is **gossiped** on **`boing/transactions`**.

Errors: `-32601` Faucet not enabled; `-32000` Faucet account not initialized or balance too low.

boing_getAccountTransactions

Committed transactions that touch an account, newest first. A transaction touches an account when the account is the **sender**, a native **transfer** recipient, a **batch step** recipient / called contract, the **called contract**, an open-validator `node_id`, or equals any **log topic** in its receipt (token / NFT transfer events, registry events). Faucet payouts are plain transfers, so they appear for the recipient.

Params	**`[hex_account]` or `[hex_account, { "cursor"?: string | null, "limit"?: number }]`**
Result	`{ account, head_height, transactions: [...], next_cursor: string | null }`

Each row: `tx_id`, `block_height`, `tx_index`, `timestamp`, `sender`, `nonce`, `direction` (`out` when the account sent it, else `in`), `roles`, `payload` (`kind` plus `to` / `amount` / `contract` / `steps`; a contract call or batch step that carries native BOING adds `value` (base units) and `value_boing`, testnet-v0.1.18+), `success`, `error`, `gas_used`.

Paging is deterministic: rows are ordered by `(block_height, tx_index)` descending; pass `next_cursor` (opaque `"<height>:<index>"`, meaning strictly older) to fetch the next page. `limit` is clamped to 1..100 (default 25). The index is built in memory from committed blocks and caught up on each call (rebuilt after a restart).

boing_getValidators

List the current validator set and (in open mode) every registered validator.

Params: `[]`

Result: `{ mode: "static"|"stake"|"open", height, quorum, validator_count, open: null | { activation_height, activated_at, bond, epoch_len, next_epoch_height, max_open_validators, liveness_window, miss_threshold_bps, jail_cooldown, ... }, validators: [ { id, kind:`

`"anchor"|"static"|"open", status: "active"|"pending"|"jailed"|"exiting", in_set, bond (string, ledger units: base units from the 18-decimals switch), bond_boing (formatted BOING), owner?, jailed_until, scheduled_proposals, missed_proposals, uptime_pct } ] }`

- Liveness counts proposals only over produced blocks in the window (votes are not on-chain); idle periods are not counted.
- The window is reset when a validator is jailed. `last_jail` (`{ jailed_at, scheduled_proposals, missed_proposals, uptime_pct }` or `null`) keeps the stats at the most recent jailing. It is node-local display data (not part of the state root); `null` means this node did not record it (e.g. jailed before the node was upgraded).
- Open-mode transactions (`ValidatorRegister` = payload 10, `ValidatorDeregister` = 11, `ValidatorUnjail` = 12) are rejected by the mempool until the mode is active.

Name registry (`.boing` names)

Native name registry (see `NAME-REGISTRY-PROPOSAL.md`). Disabled unless the node is started with `--name-registry-activation-height <H>` or `BOING_NAME_REGISTRY_ACTIVATION_HEIGHT=<H>`; before height `H` name transactions are rejected and these reads return `null` / `[]` / `"registry": {"enabled": false}`. All reads are evaluated at `head + 1` (the height a transaction submitted now would execute at). Names are normalized: trimmed, lowercased, optional `.boing` suffix stripped; 3-32 chars of `a-z 0-9 -`.

Method	Params	Result
`boing_resolveName`	`[name]`	`null` or `{ name, display, owner, target, registered_at, expires_at, status }` (only **active** names resolve; names in grace do not)
`boing_reverseName`	`[address]`	`null` or `{ name, display, address, verified: true }`; only returned when the primary name is active and still resolves to `address`
`boing_getNameInfo`	`[name]`	`{ input, name, display, valid, reserved, available, status, rent_per_year, record?, registry, head_height }`
`boing_checkNames`	`[[name, ...]]` (1-25)	`{ registry, results: [ <same shape as getNameInfo without record> ] }`
`boing_namesOwnedBy`	`[address]`	array of records (active and grace) owned by `address`

Name transactions (payload variants 13-19, all sender-only access list, base gas 21,000): `NameCommit{commitment}`, `NameRegister{name, salt, target, years}`, `NameRenew{name, years}`, `NameTransfer{name, new_owner}`, `NameSetTarget{name, target}`, `NameSetPrimary{name|null}`, `NameRelease{name}`. Registration is commit-reveal: `commitment = blake3("BOING_NAME_COMMIT_V1" || u32le(len) || label || owner || salt)`; the register must land at least `commit_min_age` and at most

`commit_max_age` blocks after the commit. Rent (burned): 10 / 5 / 2 / 1 BOING per year for 3 / 4 / 5 / 6+ character labels, 1-10 years. After `expires_at` a name has a grace period (`grace_blocks`) during which only the owner can renew; then anyone can register it.

CLI: `boing name resolve|reverse|info|check|owned|claim|renew|transfer|set-target|set-primary|release` (`claim` runs commit → register, optionally `--primary` ; signing commands take `--secret-hex` or `BOING_SECRET_HEX` , and `--dry-run`). SDK: `boing-sdk` `boingNames` helpers (`resolveBoingRecipient` , `commitBoingName` , `registerBoingName` , ...) and `BoingClient.resolveName` / `reverseName` / `getNameInfo` / `checkNames` / `namesOwnedBy` .

Priority tips and block signatures (stable spec)

Machine-readable copy: `boing_getRpcMethodCatalog` -> `protocol_specs` (source `crates/boing-node/schemas/developer_api.json`). These rules are stable and additive-only.

Priority tips

- **Units:** `priority_tip` is a `u128` in ledger units (base units; 1 BOING = 10^18). In JSON it is a decimal string.
- **Wire:** `SignedTransaction.priority_tip` is appended after the signature as 16 bytes little-endian and is **omitted entirely when `0`**, so untipped transactions keep their historical bytes. Always decode with `SignedTransaction::from_bincode`, never a plain bincode deserialize.
- **Signing digest:** `priority_tip == 0` uses the v2 digest (`BOING_TX_V2`). Otherwise `BLAKE3("BOING_TX_PRIORITY" || chain_id_le8 || nonce_le8 || sender32 || bincode(payload) || bincode(access_list) || tip_le16)`. The tip is signed, so a relay cannot add or change it. Golden vectors: `fixtures/tx-signing-golden-vectors.json`.
- **Admission:** balance must cover the failed-tx fee plus the tip (`insufficient_fee`).
- **Settlement:** split by the fee policy (`tip_proposer_bps`, `tip_treasury_bps`, `tip_burn_bps` in `boing_getNetworkInfo.treasury.fee_policy`); default 100% to the block proposer.
- **Ordering:** block building drains nonce-contiguous runs per sender, highest effective tip first; the lowest tip is evicted first when the pool is full. See OPTIONAL-PRIORITY-FEES.md.

Block signatures

`Block` is `{ header, transactions, signatures?, priority_tips? }` (bincode); `signatures` and `priority_tips` are each omitted when empty and are **not** part of the header or block hash.

- **`signatures[i]`** is the sender's Ed25519 signature over `transactions[i]`, in the format required at that height (v2 chain-bound after the v2 activation).
- **Required from** `boing_health.tx_signing.block_signatures_from_height` (testnet 920; genesis on new chains): exactly one signature per transaction, verified by every importing node. A missing, forged, legacy-format or other-chain signature rejects the block.
- **`priority_tips[i]`** is the tip committed by `signatures[i]`; empty means every tip is `0`, otherwise its length must equal `transactions.len()`.
- Decode with `Block::from_bincode`; layouts differ across eras.

Chain metadata file

`chain-metadata.json` lists chain ids (testnet 6913, mainnet 6914 planned), native currency, explorer, faucet and RPC URLs, signing-domain pointers and where to read the live fee policy. `boing_getNetworkInfo.developer.chain_metadata_url` points at it. Live values (heights, fee policy) always

come from the node.

Error Codes

Code	Meaning
-32600	Invalid Request
-32601	Method not found
-32602	Invalid params
-32000	Server error
-32016	Rate limit exceeded
-32050	**QA: Deployment rejected** — Transaction rejected by protocol QA (e.g. bytecode or purpose rule). Response SHOULD include `data: { rule_id: string, message: string }` for structured feedback. See QUALITY-ASSURANCE-NETWORK.md.
-32051	**QA: Pending pool** — Deployment referred to governance QA pool (result: Unsure). Response includes `data: { tx_hash: string }` (hex).
-32052	**QA pool** — No pending item for the given `tx_hash`.
-32053	**QA pool** — Voter is not eligible (not a member / public-membership gate, deployer voting on their own item, or below `min_voter_stake`).
-32054	**QA pool disabled** — Governance has not enabled the pool (`max_pending_items` is 0, or none of `public_membership` / `dev_open_voting` / `administrators`).
-32055	**QA pool full** — Global `max_pending_items` reached; optional `data.reason: "pool_full"`.
-32056	**QA pool deployer cap** — Sender exceeded `max_pending_per_deployer`; optional `data.reason: "deployer_cap"`.
-32057	**Operator RPC auth** — `boing_operatorApplyQaPolicy`, `boing_operatorApplyFeePolicy`, `boing_submitSlashAppeal`, `boing_resolveSlashAppeal`, `boing_registerBlsDualKey`, `boing_putBlobSampleOffer`, or **admin-only** `boing_qaPoolVote` called without a valid `X-Boing-Operator`, **or** while `BOING_OPERATOR_RPC_TOKEN` is unset on the node (fail-closed).

Media blobs, data availability (DAS) and BLS dual keys

L1 media blobs (see BOING-L1-MEDIA-BLOBS.md for semantics, caps and phases). Most reads need the node's **media blob store** (a data directory); without one they return `-32000 "media blob store unavailable (no data dir)"`. `blobId` accepts `0x<64 hex>`, bare hex, or `boing://blob/0x...`. Handlers: `crates/boing-node/src/media_blobs_rpc.rs`.

Method	Params	Result
`boing_getMediaBlobParams`	`[]`	Caps / activation / gas knobs for L1 media blobs.
`boing_getBlobMeta`	`[blobId]`	Metadata object or `null`.
`boing_getBlobAccess`	`[blobId]`	`null` or `{ blobId, publisher, grantees[], epoch, aclActive, aclActivationHeight, note }` (Phase 34 download ACL; see HANDOFF_BLOB_DOWNLOAD_ACL.md).
`boing_getBlob`	`[blobId]`	`{ meta, dataHex }` or `null`. When **`BOING_BLOB_GET_REQUIRE_AUTH=1`** the operator / download token credentials are required (same as the HTTP download); otherwise open. Prefer **`GET /v1/media/blob?id={blobId}`** for large raw downloads.
`boing_getBlobDasSample`	`[blobId, chunkIndex]`	`{ blobId, chunkIndex, chunkCount, chunkHash, chunksRoot, siblings[], parityChunkHex, originalLen }` (Merkle sample; does not return the data chunk). **Not** covered by `BOING_BLOB_GET_REQUIRE_AUTH`; the node re-reads the blob per call, so front public nodes with rate limits.
`boing_getBlobDasLightClientBatch`	`[blobId, sampleCount?, vrfOutput?]` or object with optional `useLeaderVrf`	Batch of samples (`sampleCount` 1.. `PHASE6_MAX_DAS_LIGHT_CLIENT_SAMPLES`, default 8) with `seedMode` = `vrf` / `leaderVrf` / `height`. The `StubBlake3V0` proof is a hash stub, **not** a SNARK.
`boing_getBlobDasSuccinctProofPlan`	`[]`	Supported proof kinds (`stubBlake3V0`, `reservedRealV1`), size caps and phase notes.
`boing_verifyBlobDasSuccinctProof`	object `{ blobId, sampleCount?, ... proof }`	Verification result; `reservedRealV1` is rejected until a real succinct system is wired.
`boing_getLeaderVrfOutput`	`[round?]` (default: chain height)	`{ round, leaderElection, vrfOutput|null, note }`; `null` under RoundRobin election.
`boing_putBlobSampleOffer`	`{ blobId, chunkIndex, chunkCount, provider, askFee, expiresAtHeight? }`	Adds a **node-local** (non-consensus) sample offer. **Operator-gated** (`-32057` without a valid `X-Boing-Operator`, or when `BOING_OPERATOR_RPC_TOKEN` is unset on the node).

Method	Params	Result
`boing_listBlobSampleOffers`	`[blobId, chunkIndex?, source?]` (`source` = `all` / `local` / `onChain`)	Offers from the local book and the on-chain registry.
`boing_listSmtBlobSampleOffers`	`[blobId, chunkIndex?]`	Consensus (SMT-resident) offers with `registeredAtHeight`, `escrowedAsk`.
`boing_matchBlobSampleOffer`	`{ blobId, chunkIndex, ... }`	Best matching offer (on-chain preferred, then local) or `-32000 "no matching sample offer"`.
`boing_quoteBlobSampleSettle`	`{ blobId, chunkIndex, protocolSampleFee? }`	`{ source, offer, quote }` settle quote for the best offer.
`boing_listBlobSampleServeChallenges`	`[blobId]` or `{ blobId?, status?, includeTerminal? }`	Phase 24 serve challenges.
`boing_getBlobSampleServeChallenge`	`{ blobId, chunkIndex, provider, challenger }`	One serve challenge or `null`.
`boing_listBlsDualKeys`	`[]`	`{ count, file, wired, popRequired, liveTx32StillEd25519, entries: [{ accountId, blsPubkey }] }`.
`boing_registerBlsDualKey`	`[accountId, blsPubkey, blsPop, chainId?]` or object (`accountId`, `blsPubkey`, `blsPop`)	Registers a BLS key with proof-of-possession. **Operator-gated** (`-32057` without a valid `X-Boing-Operator`, or when `BOING_OPERATOR_RPC_TOKEN` is unset on the node).
`boing_verifyBlsAggregate`	object	Dry-run verification of a BLS aggregate (library/RPC only; not consensus-admitted).
`boing_verifyBlsAggregateResolve`	object	Dry-run of an aggregate-attested slash-appeal resolve (tx 33 is **not** consensus-admitted; committee nets use tx 32).

HTTP media download (non-JSON-RPC): `GET /v1/media/blob?id={blobId}` returns raw bytes (`Content-Disposition: attachment`). Auth: operator/download shared secrets (`X-Boing-Operator` / `X-Boing-Blob-Download` / `Authorization: Bearer`) and/or Phase 34 publisher/grantee proof headers (`X-Boing-Blob-Downloader` , `Expires` , `Nonce` , `Signature`). `BOING_BLOB_ACL_ACTIVATION_HEIGHT` makes download private-by-default. See BOING-L1-MEDIA-BLOBS.md and HANDOFF_BLOB_DOWNLOAD_ACL.md.

Boing Network — Quality, enforced by the protocol.